

PW EARNERS
DSA in C++

Daily Practice Problems (DPP) — 9

Topic: Strings

Multiple Choice Questions (Theory & Predict the Output)

For Q1 to Q8: Select the correct option. Assume all required headers like `<iostream>`, `<string>`, and `<algorithm>` are included, and `using namespace std;` is present.

Q1. What will be the output of the following C++ code snippet?

```
1 int main() {  
2     string s = "cba";  
3     sort(s.begin(), s.end());  
4     cout << s;  
5 }
```

- A. cba B. abc C. Compilation Error D. Garbage Value

Q2. What is the time complexity of converting an integer to a string using the `to_string()` function?

- A. $O(1)$ C. $O(N)$ where N is the integer's numeric value
B. $O(\text{len})$ where len is the number of digits D. $O(N \log N)$

Q3. Predict the output of the following code snippet:

```
1 int main() {  
2     string str = "Arjun is the smartest boy";  
3     cout << str.substr(6, 2);  
4 }
```

- A. is B. in C. th D. ar

Q4. Predict the output of the following string manipulation:

```
1 int main() {  
2     string s = "apple";  
3     s[0] = 'A';  
4     cout << s;  
5 }
```

- A. Apple B. apple C. A D. Compilation Error

Q5. What is the optimized time complexity for finding the most frequent character in a string using a frequency array mapping?

- A. $O(N^2)$ B. $O(N \log N)$ C. $O(N)$ D. $O(1)$

Q6. Which of the following is the correct function to convert a string object representing a large number (e.g., "9172600000") into a `long long` integer in C++?

- A. `stoi()` B. `stoll()` C. `to_integer()` D. `stol()`

Q7. Predict the output of the following code snippet:

```
1 int main() {  
2     int n = 98765;  
3     cout << to_string(n).length();  
4 }
```

- A. 98765 B. 0 C. 6 D. 5

Q8. When trying to find the longest continuous word in a string containing spaces, which algorithmic technique is most optimal and standardly recommended?

- A. Sorting the entire string alphabetically C. Variable-size Sliding Window (2-Pointer Technique)
B. Nested loops checking every character globally D. Prefix Sum Array

For Q9 to Q15: Write clean, optimal, and compilable C++ programs. The questions are arranged in increasing order of difficulty.

Q9. Defanging an IP Address. Given a valid (IPv4) IP address as a string, return a defanged version of that IP address. A defanged IP address replaces every period "." with "[.]". Solve it in-place or by building a new string in $O(N)$ time.

Constraints:

$7 \leq \text{address.length} \leq 15$

`address` is a valid IPv4 address.

Sample Input: `address = "1.1.1.1"`

Sample Output: `"1[.]1[.]1[.]1"`

Q10. Remove All Adjacent Duplicates In String. You are given a string `s` consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them. We repeatedly make duplicate removals on `s` until we no longer can. Return the final string after all such duplicate removals have been made.

Constraints:

$$1 \leq s.length \leq 10^5$$

`s` consists only of lowercase English letters.

Sample Input 1: `s = "abbaca"`

Sample Output 1: `"ca"`

Explanation:

For example, in `"abbaca"` we could remove `"bb"` since the letters are adjacent and equal. The string then becomes `"aaca"`. We then remove `"aa"`, which leaves `"ca"`. No more adjacent duplicates exist.

Sample Input 2: `s = "azxxzy"`

Sample Output 2: `"ay"`

Explanation:

Remove `"xx"` to get `"azzy"`. Then remove `"zz"` to get `"ay"`.

Q11. Longest Common Prefix. Write a function to find the longest common prefix string amongst an array of strings. If there is no common prefix, return an empty string `""`.

Constraints:

$$1 \leq \text{strs.length} \leq 10^4$$

$$0 \leq \text{strs}[i].\text{length} \leq 100$$

`strs[i]` consists of only lowercase English letters.

Sample Input: `strs = ["flower", "flow", "flight"]`

Sample Output: `"fl"`

Q12. Check String Rotation. Given two strings `s` and `goal`, return `true` if and only if `s` can become `goal` after some number of shifts on `s`. A shift on `s` consists of moving the leftmost character of `s` to the rightmost position.

Constraints:

$$1 \leq s.length, goal.length \leq 10^5$$

`s` and `goal` consist of lowercase English letters.

Sample Input: `s = "abcde", goal = "cdeab"`

Sample Output: `true`

Explanation:

Shifting `s` ("abcde") one position to the left gives "bcdea". Shifting it one more position to the left gives "cdeab", which matches `goal`.

Q13. Absolute Frequency Difference of Characters. Given two strings `s1` and `s2` consisting of lowercase English letters, generate a new string based on the characters that differ in frequency between the two strings. Specifically, calculate the absolute difference in the frequency of each character. If a character appears x times in `s1` and y times in `s2`, it should appear $|x - y|$ times in the output string. The final string must be printed in strictly alphabetical sorted order.

Constraints:

$1 \leq \text{s1.length}, \text{s2.length} \leq 10^5$

`s1` and `s2` consist of lowercase English letters.

Sample Input 1: `s1 = "hello", s2 = "world"`

Sample Output 1: `"dehlrw"`

Explanation:

'l' is present 2 times in `s1` and 1 time in `s2`, difference is 1. 'h' and 'e' are in `s1` but not `s2`, difference is 1 each. 'w', 'r', 'd' are in `s2` but not `s1`, difference is 1 each.

Sample Input 2: `s1 = "abc", s2 = "abc"`

Sample Output 2: `""`

Explanation:

All characters have the same frequency in both strings, so the difference is 0 for all characters.

Sample Input 3: `s1 = "aab", s2 = "bcc"`

Sample Output 3: `"aabcc"`

Explanation:

'a' appears 2 times in `s1` and 0 times in `s2` (diff = 2). 'b' appears 1 time in both strings (diff = 0). 'c' appears 0 times in `s1` and 2 times in `s2` (diff = 2). The output string combines these alphabetically.

Q14. Sort Characters By Frequency. Given a string `s`, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string. Return the sorted string. If multiple answers exist, return any of them.

Constraints:

$$1 \leq s.length \leq 5 \times 10^5$$

s consists of uppercase and lowercase English letters and digits.

Sample Input: s = "tree"

Sample Output: "eert" (or "etr")

Q15. Group Anagrams. Given an array of strings `strs`, group the anagrams together. You can return the answer in any order. An Anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Note: Since you have not been taught Hashing/Maps yet, a less optimized approach (using sorting and arrays/vectors) is completely acceptable. You will learn to solve this in a highly optimized way later!

Constraints:

$$1 \leq strs.length \leq 100$$

$$0 \leq strs[i].length \leq 50$$

`strs[i]` consists of lowercase English letters.

Sample Input: `strs = ["eat", "tea", "tan", "ate", "nat", "bat"]`

Sample Output: `[["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]`

All the best! Keep Coding ♡